

Dónde estamos, cómo funciona, y qué falta para migrar

Un recorrido con ejemplos reales, corridos contra el código de hoy.
Ninguna cifra de este documento está inventada.

470 pruebas

Todas en verde. Se vuelven a correr con una orden.

45 / 46

Hallazgos de auditoría cerrados con prueba.

20 decisiones

Pendientes de que alguien con autoridad las firme.

0 migrados

Todavía no se ha tocado una sola cuenta real.

Lo más importante de esta portada. El sistema está construido y probado, pero probado *en aislamiento*: contra datos inventados, en una máquina, sin red y sin dinero. Eso no es lo mismo que probado en producción. La página 7 explica la diferencia, y es la razón por la que todavía no migramos.

1 · Dónde estamos, en una página

Hay un árbol de código nuevo, separado de todo lo que hoy está en producción, que se llama `sfsp/`. No toca nada de lo que funciona. Se puede borrar entero hoy y no se cae nada.

Dentro hay cinco piezas, y cada una tiene sus propias pruebas:

PIEZA	PRUEBAS	QUÉ HACE
sdk	125	Las reglas: cuentas, números, alias, rutas, reservas, suministro, migración. Es el cerebro.
contracts	178	Lo que vive en la cadena: activos, gobierno, identidad, liquidación, migración.
indexer	67	Lee la cadena y la vuelve consultable. Sabe deshacer una reorganización.
dbnx-api	86	Admisión de activos: expedientes, firmas, informes.
adversarias	16	Pruebas escritas para <i>romper</i> lo anterior. Una por cada defecto que encontró la auditoría.

El semáforo honesto

COSA	ESTADO	QUÉ SIGNIFICA
El código	LISTO	Escrito, probado y auditado por fuera.
Los números económicos	VACÍOS	A propósito. Ni un precio, ni un ratio, ni un quórum está escrito en el código.
Las decisiones	20 PENDIENTES	Nadie con autoridad ha firmado todavía. Sin firma no hay número.
Los accesos	NO TENEMOS	Ni a la red real, ni a los servidores, ni a las llaves.
Prueba en la red real	CERO	Nada de esto ha visto la red 5550 ni un saldo de verdad.
Migración	NO EMPEZADA	Ninguna cuenta real ha sido tocada.

En cristiano: **tenemos el motor armado y probado en el banco de pruebas. No lo hemos montado en el carro, y el carro todavía no tiene dueño que firme el papel.**

2 · Qué es SFSP, y qué no es

Qué Sí es

- Una capa de **reglas sobre activos** que se apoya en la cadena que ya tenemos.
- Un **directorio**: convierte un número de cuenta en una dirección de billetera.
- Un **portero**: decide si una operación se permite, y siempre dice por qué no.
- Un **libro de cuentas de la migración**: qué se registró, qué quedó fuera y si cuadra.

Qué NO es

- **No es una cadena nueva**. No es un fork. La red 5550 sigue siendo la misma.
- **No es un cambio de consenso**. Los validadores no se enteran.
- **No mueve saldos por sí solo**. La migración registra, no traslada.
- **No es un banco**. No promete respaldo, ni rendimiento, ni recuperación de fondos.

Las cuatro cosas que todo el mundo confunde

Esta tabla es la que más vas a necesitar. Son **cuatro cosas distintas**, y tratarlas como una sola es el origen de la mitad de los problemas que puede tener una migración.

COSA	EJEMPLO	PARA QUÉ SIRVE Y QUIÉN LA VE
Cuenta SFSP	SF-0324-8432-6581-4	Lo que le das a alguien para que te pague. Pública . No da ningún poder sobre el dinero.
Genesis ID	gsr_...	La identidad de la persona dentro del sistema. Interna. No se publica.
Dirección de billetera	0x7099...79C8	Dónde está el saldo de verdad, en la cadena. Pública, pero el usuario no tiene que verla.
Llave privada / semilla	(nunca se escribe)	El poder de gastar. Secreta . El sistema está diseñado para no necesitarla nunca.

Por qué importa para la migración. Como la cuenta SFSP y la billetera son cosas distintas, **podemos darle a alguien un número de cuenta nuevo sin tocarle ni un centavo**. Su ONDK se queda exactamente donde está. Eso es todo el truco, y está explicado en la sección 8.

3 · Cómo nace una cuenta, de verdad

Esto no es un dibujo. Es la salida real de correr el código hoy, sin retocar:

```
$ node demo.mjs

ALTA:      SF-0324-8432-6581-4  ← el número que se le da a la persona
estado:    PENDING              ← existe, pero TODAVÍA NO RECIBE DINERO

... alguien con autoridad aprueba el alta ...
ACTIVADA:  ACTIVE

ALIAS:     @medardo
RUTA:      red 5550 → 0x7099...79C8 · propósito PAYMENTS

RESUELVE A: {
  "accountNumber": "SF-0324-8432-6581-4",
  "address":       "0x70997970C51812dc3A010C7d01b50e0d17dc79C8",
  "expiraEnISO":   "2026-09-22T16:01:27.817Z" ← caduca en 60 segundos
}
```

Tres detalles que parecen pequeños y no lo son

1 La cuenta nace apagada.

Nace en `PENDING` y en ese estado

no resuelve

: si alguien intenta pagarle, el sistema rechaza. Hay que activarla a propósito, con un nombre y un motivo escritos. Esto era un defecto hasta hace unos días: las cuentas nacían activas, es decir, se daba por aprobada un alta que nadie había aprobado. La auditoría lo encontró y está corregido.

2 El número no es correlativo.

`SF-0324-8432-6581-4` no viene después de `SF-0324-8432-6581-3`. Son doce dígitos al azar de verdad más un dígito de control. Nadie puede adivinar cuántos clientes tenemos ni tantear cuentas ajenas. Y un número usado

no se recicla jamás

, ni aunque la cuenta se cierre, porque alguien pudo haberlo apuntado en su agenda.

3 La resolución caduca en 60 segundos.

Cuando el sistema dice «esa cuenta es esta dirección», esa respuesta lleva fecha de vencimiento. Si alguien la guarda y la usa media hora después, ya no vale. Así, cambiar de billetera no deja pagos volando hacia la billetera vieja.

4 · El alias, y por qué rechaza nombres parecidos

Un alias es el nombre bonito: `@medardo` en vez de un número de catorce dígitos. El peligro obvio es que alguien registre algo que se vea igual y cobre lo que no es suyo. Esta es la salida real de intentarlo:

```
$ node alias.mjs      # @medardo ya lo tiene otra persona

"@medardo"    → RECHAZADO ese alias ya está tomado
"@MEDARDO"    → RECHAZADO ese alias ya está tomado      ← mayúsculas no cuentan
"@medardo "   → RECHAZADO ese alias ya está tomado      ← el espacio tampoco
"@medard0"    → RECHAZADO ya existe un alias que se ve igual ← cero por o
"@rnedardo"   → RECHAZADO ya existe un alias que se ve igual ← "rn" por "m"
```

Los dos últimos son los importantes. `@medard0` con cero y `@rnedardo` con «rn» son cadenas de texto **distintas** — un sistema ingenuo las acepta sin dudar. SFSP calcula el «esqueleto» de cada nombre, que es cómo se ve una vez que se le quitan las diferencias que el ojo no capta, y compara por ahí.

Y hay dos índices distintos en la base de datos, no uno: el primero impide el mismo alias, el segundo impide uno que se vea igual. Que el segundo nunca dispare es la prueba de que el primero funciona.

5 · Qué pasa cuando alguien intenta pagar

- 1 Llega un destino**
Puede ser el número `SF-...` o el alias `@medardo`. Los dos caminos llegan al mismo sitio.
- 2 ¿La cuenta está activa?**
Si está en `PENDING`, `SUSPENDED` o `CLOSED`, se acabó ahí. El estado no es una etiqueta: le cierra la puerta al dinero.
- 3 ¿Tiene una ruta vigente en esa red?**
Una cuenta puede tener varias billeteras. Se elige la marcada como principal, y sólo si está vigente hoy.
- 4 ¿La política lo permite?**
Aquí entran identidad, jurisdicción, estado del activo y límites. Es el portero.
- 5 Se entrega una respuesta con fecha de caducidad**
Y quien la ejecute debe volver a comprobar que nada cambió entre medias.

Las respuestas posibles, y la regla de oro

RESPUESTA	CUÁNDO
ALLOW	Adelante.
DENY_POLICY	La política dice que no.
DENY_ELIGIBILITY	Esa persona no cumple los requisitos para este activo.
DENY_JURISDICTION	No en ese país.
DENY_ASSET_STATE	El activo está pausado, congelado o no es transferible.
DENY_AUTHORIZATION	Falta una firma, o la que hay no vale.
DENY_LIMIT	Se pasa de un tope.
REVIEW_REQUIRED	Hace falta que lo mire una persona.
UNKNOWN_SOURCE	No sé. Un dato no llegó, o llegó roto.

La regla de oro del sistema. UNKNOWN_SOURCE **nunca** se convierte en cero ni en «permitido». Si el sistema no sabe, dice que no sabe y se detiene. Suena obvio; es justo lo que falla en la mayoría de los sistemas de pago, donde un dato que no llega se trata como un cero y un cero parece que está todo bien.

6 · Nadie firma solo

Toda acción que puede hacer daño exige **dos personas distintas**: una propone y otra aprueba. Quien propone **no cuenta** como aprobador, aunque tenga permiso para aprobar.

Y la autorización no es un permiso suelto que después se aplica a cualquier cosa. Es una huella que compromete *todo*: qué acción, sobre qué activo, entre quiénes, por cuánto, en qué red, en qué contrato, hasta cuándo, y con qué número de serie. Quien ejecuta **recalcula esa huella desde los argumentos reales** y la gasta una sola vez.

Esto arregla de una sola vez **siete de los nueve defectos más graves** que encontró la auditoría. Todos eran la misma cosa: una autorización identificada por algo débil (un nombre, un número de operación) que después se aplicaba a otra cosa distinta de la que se aprobó.

Las diez acciones, todas con doble control

MINT emitir · BURN quemar · FORCED_TRANSFER transferencia forzosa · UNPAUSE reanudar · SETTLE_DVP liquidar · UPGRADE actualizar · RECOVERY recuperar · SET_QUORUM cambiar quóruns · SET_POLICY cambiar política · MIGRATION_CLAIM reclamo de migración.

Hay una prueba que las recorre las diez contra el ejecutor real y comprueba tres cosas en cada una: que quien propone no puede aprobar, que con un solo aprobador la operación se rechaza, y que después del rechazo **el estado no se movió**.

El caso más delicado: cambiar los quóruns

SET_QUORUM es la acción que cambia cuántas firmas hacen falta para las demás. Si se pudiera usar para rebajarse a sí misma, una sola persona podría abrirse la puerta. Tiene dos candados, con una prueba cada uno:

- **En la misma operación**: lo que valida el cambio es el número *anterior*, no el nuevo.
- **Encadenada**: bajar el quórum hoy no abarata una autorización que ya estaba propuesta. El exigible es el *mayor* entre el congelado al proponer y el vigente.

La única excepción, y por qué

La **pausa de emergencia** la puede echar una sola persona. Es deliberado: una pausa que tuviera que reunir quórum llegaría tarde, y para entonces el daño está hecho. El contrapeso es que **caduca sola** y que **levantarla sí exige doble control** atado al incidente. Es más fácil parar que arrancar, que es como debe ser.

7 · Qué probamos, y qué no · la página más importante

El sistema no dice «probado» o «sin probar». Tiene seis niveles, y sabe en cuál está cada cosa. Esto existe para que nadie —yo incluido— pueda decir que algo está probado cuando no lo está.

NIVEL	QUÉ SIGNIFICA DE VERDAD
DECLARADO	Alguien lo escribió en un documento. No hay código.
CODIGO_LEIDO	Hay código y alguien lo leyó. No se ejecutó.

PROBADO_AISLADO	Aquí estamos hoy. Se ejecuta y pasa, contra datos inventados, en una máquina, sin red.
VERIFICADO_RUNTIME	Funciona en un servicio desplegado, contra la red real.
NO_VERIFICADO	Se intentó y no se pudo comprobar.
BLOQUEADO	No se puede comprobar porque falta un acceso o una decisión.

La distancia que separa hoy de la migración. Estamos en `PROBADO_AISLADO`. Para migrar hace falta `VERIFICADO_RUNTIME`. Entre los dos hay: accesos que no tenemos, decisiones que nadie ha firmado, y un entorno de pruebas aislado que todavía no existe. **Ninguna de las tres las resuelve escribir más código.**

Cómo se comprueba, y qué salió mal en el camino

Hay una orden que corre las 470 pruebas, comprueba los tipos, cuenta las decisiones pendientes, verifica que no haya ni un número económico escrito, y **firma un archivo de evidencia con el commit exacto del código que probó.**

```
$ node scripts/verificar-todo.mjs

sdk                PROBADO_AISLADO    125 pruebas
contracts          PROBADO_AISLADO    178 pruebas
indexer            PROBADO_AISLADO     67 pruebas
dbnx-api           PROBADO_AISLADO     86 pruebas
pruebas-adversarias PROBADO_AISLADO     14 pruebas

árbol limpio: sí
decisiones pendientes: 20
parámetros económicos con valor: 0

cobertura: VERIFICACION_COMPLETA
estado:    PROBADO_AISLADO
```

Una cosa que te debo contar, porque es la que más me preocupa de todo este trabajo. Ese «árbol limpio: sí» estuvo *mintiendo* desde que escribí la herramienta. Preguntaba por una carpeta con la ruta mal puesta; el sistema devolvía una lista vacía sin quejarse, y la lista vacía se leía como «no hay nada sucio». O sea: **la herramienta escrita para impedir que firmáramos evidencia de un árbol sin guardar llevaba meses firmando que estaba limpio sin haber mirado nunca.** Lo encontré ayer, al ver que decía «limpio» con dos documentos sin guardar delante. Está corregido, y ahora «no hay nada sucio» y «pregunté mal» son dos respuestas distintas. Toda la evidencia anterior vale por lo que probó, pero no por su afirmación de procedencia.

8 · La migración · registrar sin mover

Esta es la sección que te importa. La pregunta que planteaste desde el principio era: **¿cómo hacemos esto sin que nadie se dé cuenta y sin que nadie pierda sus ONDK ni ninguna otra moneda?**

La respuesta es una sola frase: **no movemos nada.**

LA IDEA COMPLETA

Los saldos se quedan exactamente donde están, en las mismas billeteras, en la misma cadena. Lo único que hacemos es **anotar en un directorio nuevo** que la billetera `0x...1001` ahora también responde al número `SF-8375-3040-2151-0`. Ratio 1 a 1. Cero transacciones sobre el saldo. Cero riesgo de que un traslado se caiga a la mitad.

Por qué esto es seguro, y por qué la alternativa no lo es

Mover los saldos

- Miles de transacciones que pueden fallar a la mitad.
- Hay que pagar el gas de todas.
- Hay que tener las llaves de todo el mundo.
- Si se cae a medias, unos tienen y otros no.
- El usuario ve movimiento y se asusta.

Registrar sin mover

- El saldo nunca se toca. No hay nada que se caiga.
- No hace falta gas para los saldos.
- **No hace falta la llave de nadie.**
- Se puede parar a la mitad sin dejar a nadie colgado.
- El usuario no ve nada: gana un número, no pierde saldo.

Corrido de verdad, con el censo de prueba

Cuatro cuentas inventadas, dos con saldo en `ORIGEN` y en `ONDK`. Esta es la salida real:

```
— PASO 1 · SIMULACRO (ensayo general, no escribe nada) —
cuentasOrigen:      4
cuentasSFSP:        4
saldosIguales:      true      ← nadie ganó ni perdió nada
excepciones:        0
cuadra:             true

cuentas creadas de verdad en el directorio: 0  ← el ensayo NO dejó rastro

— PASO 2 · APLICAR (esta vez sí) —
pares: 4 · cuadra: true · saldosIguales: true
  sint_0001 → SF-8375-3040-2151-0
  sint_0002 → SF-6369-2926-3231-5
  sint_0003 → SF-5730-5873-0919-3
excepciones: []
cuentas en el directorio: 4

— PASO 3 · CORRERLO OTRA VEZ POR ERROR —
yaExistentes: 4 · total sigue en: 4  ← nadie recibió una segunda cuenta
```

Los tres seguros que acabas de ver funcionando

1 El ensayo no deja rastro.

Fíjate en «cuentas creadas de verdad: 0» después de un simulacro que reportó 4. Esto también era un defecto: el simulacro escribía en el directorio de verdad, así que ensayar la migración *era* migrar. Corregido: el simulacro trabaja sobre una copia.

2 Correrlo dos veces no hace daño.

Si el proceso se cae y alguien lo vuelve a lanzar, las cuentas que ya tenían número no reciben otro. Suena menor; es la diferencia entre un susto y un desastre, porque en una migración real el proceso *se va a caer* alguna vez.

3 Nadie se queda fuera en silencio.

Si una cuenta no se puede migrar,
no se borra del censo

: se abre un expediente con el motivo. Y el contador `direccionesConSaldoSinDueno` existe precisamente para gritar si apareciera una billetera con dinero que no le corresponde a nadie.

La cuadratura: la prueba de que nadie perdió nada

Al final de cada corrida se comprueba una suma que no puede fallar:

$$S0 = A + N + P$$

todo lo que había = lo migrado + lo no reclamado + lo pendiente

```
CUADRA:      true  diferencia: 0
... ahora le quito UNA unidad de mil millones a lo migrado ...
NO CUADRA:   false diferencia: 1  ← lo pilló al instante
```

Una sola unidad de diferencia y la corrida se marca como no cuadrada. No hay tolerancia, no hay redondeo, no hay «casi». Y hay una segunda igualdad, $E = N + P$, que comprueba lo mismo desde el otro lado, para que un error tenga que mentir dos veces de forma coherente para pasar desapercibido.

Sobre los ONDK en concreto. En el censo de prueba aparecen como `SFSP:LEGACY:OG:ONDK`, la etiqueta que marca un activo de la vida anterior. Se registran con ratio 1:1 y con su saldo intacto. Una conversión con ratio distinto de 1:1 está prevista en el código, pero **con resto explícito**: si la división no es exacta, el resto no se pierde ni se redondea a favor de nadie, se reporta.

9 · Las 20 decisiones que bloquean todo

Cada número económico del sistema vale `null` —vacío— a propósito. No es que falte escribirlos: es que **el código se niega a funcionar con un número que nadie firmó**. Si se intenta usar un parámetro cuya decisión no está aprobada, el sistema para.

Estas son las que hacen falta, ordenadas por lo que desbloquean:

Bloquean la migración directamente

ID	TEMA	QUIÉN TIENE QUE FIRMARLA
D09	Modo, ratio, gas, corte y claims de migración por activo	Junta / DBNX / tecnología
D10	Custodia canónica, excepciones y recuperación	seguridad / operaciones / legal
D18	Arquitectura final de custodia gestionada (llaves, quórum, respaldo)	seguridad / Junta
D19	Política de recuperación por activo y perfil	DBNX / legal / seguridad
D07	Quóruns, firmantes, pausa, upgrade y recuperación	Junta / seguridad
D12	Recursos de staging y aislamiento efectivo	operaciones / seguridad
D11	Repositorios y fuente de despliegue por servicio	tecnología

Bloquean el dinero y las promesas

ID	TEMA	QUÉ NO SE PUEDE HACER SIN ELLA
D01	Política de precio de ORIGEN	Conversión, valuación, tarjeta, comisiones
D02	Objetivo de USD 0.01, mínimos, patrocinio de gas	Activar un cobro nuevo
D03	Suministro nativo: respaldo total o liberación controlada	Afirmar respaldo, liberar tesorería
D04	Reservas elegibles, descuentos, concentración	Capacidad de reserva distinta de cero
D05	Custodia y redención de AUKA / AGK	Serie redimible, emisiones nuevas
D15	Proveedor de pagos, contratos, liquidez	Modo real en tarjeta y punto de venta

Bloquean lo legal y lo público

ID	TEMA	QUÉ NO SE PUEDE HACER SIN ELLA
D08	Clasificación, derechos y elegibilidad por activo y país	Ofrecer, cambiar derechos, negociar
D13	Identidad, jurisdicción, permisos y alcance regulatorio	Toda actividad que requiera licencia
D06	Alcance y diseño de privacidad	Cualquier promesa de privacidad
D17	Política de alias: reservados, disputas, recuperación	Activar los alias en público

D00	Nombre SFSP y limpieza de marca	Usar el nombre en público
D14	Retiro de la cadena 8532 y continuidad histórica	Apagar o descartar nada
D16	Resolver la especificación OGFP referida	Cerrar contradicciones normativas

Lo que esto significa en la práctica. Si mañana tuviéramos todos los accesos, **todavía no podríamos migrar**, porque el sistema se negaría a usar un ratio que nadie firmó. Eso es a propósito y es lo que impide una migración con un número puesto a ojo.

10 · El camino de aquí a la migración

- 1

Firmar las siete decisiones que bloquean la migración
D07, D09, D10, D11, D12, D18, D19. No hace falta código; hacen falta firmas. Es lo único de esta lista que se puede empezar hoy mismo.
- 2

Conseguir accesos y levantar un entorno aislado
Una copia de la red donde se pueda romper todo sin consecuencias. Esto es D12.
- 3

Que un tercero corra las pruebas desde cero
Es el único punto de auditoría que sigue abierto (P11). Están escritas las cinco órdenes para hacerlo. No lo cierro yo: cerrarlo es que lo corra otro.
- 4

Desplegar en el entorno aislado y pasar a VERIFICADO_RUNTIME
Aquí es donde dejamos de creer en las pruebas y empezamos a creer en los hechos.
- 5

Censo real y simulacro contra datos reales
El simulacro no escribe. Se corre las veces que haga falta hasta que cuadre y el número de expedientes sea explicable uno por uno.
- 6

Migrar un lote pequeño, con nombre y apellido
Cuentas nuestras, no de clientes. Comprobar que los saldos siguen intactos y que los pagos resuelven.
- 7

Migrar por lotes, con la cuadratura verde en cada uno
Cualquier lote que no cuadre detiene todo. Sin excepciones y sin «es sólo una unidad».
- 8

Fusionar el árbol nuevo con el que está en producción
Al final, no al principio. El código está preparado para extraerse limpio.

Lo que puede salir mal, y qué lo para

RIESGO	QUÉ LO DETIENE
Alguien pierde saldo	Nunca se mueve un saldo. Y la cuadratura pilla una unidad de diferencia.
Dos cuentas con el mismo número	Índice único en la base + tabla de números consumidos que nunca se borra. Probado con hilos reales peleando por el mismo archivo.
Alguien recibe dos cuentas	Una cuenta de origen tiene una sola cuenta SFSP, impuesto por la base, no por el código.
El proceso se cae a la mitad	Volver a lanzarlo no duplica nada. Sólo una corrida a la vez por censo.
Un pago va a la billetera vieja	La resolución caduca en 60 segundos y se revalida antes de ejecutar.
Alias parecido que roba pagos	Comparación por esqueleto, con dos índices distintos.
Alguien firma solo algo grave	Doble control en las diez acciones, con el proponente excluido.
Se migra con un número inventado	El sistema se niega a usar un parámetro cuya decisión no esté firmada.

11 · Lo que no te voy a maquillar

Tres cosas que un informe optimista omitiría, y que prefiero que sepas antes de decidir nada:

- 1 La herramienta de verificación me falló a mí.**
Durante todo este tiempo firmó «árbol limpio» sin haber mirado nunca. Está corregido y ahora hay una prueba que lo vigila, pero el hecho es que la herramienta escrita para garantizar la procedencia de la evidencia tenía exactamente el fallo que existía para impedir. Si eso pasó una vez, puede haber pasado otra en algo que todavía no he encontrado. Por eso el punto P11 —que un tercero corra todo desde cero— importa de verdad y no es un trámite.
- 2 La privacidad tiene un límite escrito.**
Dos billeteras de la misma persona ya
no
se pueden enlazar entre propósitos distintos —que es lo que la identificaba a través de sus cuentas—. Pero *dentro* de un mismo propósito sí se puede comprobar que son la misma. Es deliberado, porque es lo que permite tratarlas como un solo sujeto. Cerrarlo del todo cambia el modelo de ejecución entero y no se hizo.
- 3 Nada de esto ha visto dinero real.**
470 pruebas en verde contra datos inventados no son 470 pruebas contra la red 5550. La diferencia entre esas dos cosas es donde se pierden los proyectos, y es exactamente por eso que el sistema se niega a llamarlo `VERIFICADO_RUNTIME`.

12 · Si sólo te quedas con una página

RESUMEN

Lo que está hecho: el sistema completo, escrito, probado con 470 pruebas, auditado por fuera, con 45 de 46 hallazgos cerrados con una prueba que se puede volver a correr.

Lo que falta: que alguien firme 20 decisiones, que consigamos accesos, que un tercero lo verifique, y que se pruebe contra la red real.

Cómo migraremos: sin mover un solo saldo. Cada persona gana un número de cuenta; su ONDK se queda donde está, con ratio 1:1. Ensayo que no deja rastro, se puede repetir sin daño, y una suma que no admite ni una unidad de diferencia.

Lo siguiente, hoy: las siete decisiones que bloquean la migración —D07, D09, D10, D11, D12, D18, D19—. No hace falta escribir una línea de código para empezar por ahí.

Todos los ejemplos de este documento son salida real del código, corrida el 22 de septiembre de 2026. Los datos son sintéticos: ninguna dirección, saldo ni referencia corresponde a una cuenta real. Fuente: `sfsp/docs/DONDE-ESTAMOS.html`.